

26-pragmatic-programming

26. Pragmatic Programming

Level: □ Intermediate

Jam: 8 (4 minggu × 2 sesi)

Prasyarat: Dasar TypeScript/JavaScript, familiar dengan code editor

Output: Codebase yang sudah direfactor dengan prinsip-prinsip pragmatic programming

Tujuan Pembelajaran

Setelah modul ini, kamu bisa:

- Menerapkan prinsip DRY, KISS, YAGNI, Boy Scout Rule, dan SRP dalam kode sehari-hari
- Menulis kode dengan naming convention yang konsisten dan mudah dibaca
- Melakukan refactoring dengan pola-pola yang terbukti (extract function, rename, decompose conditional)
- Memberi dan menerima code review secara konstruktif dengan checklist yang sistematis
- Memperkirakan effort tugas dengan teknik PERT, planning poker, dan reference task
- Menghilangkan magic numbers dan meningkatkan maintainability kode

Materi

Sesi	Topik	File
1	Clean Code Basics — DRY, KISS, YAGNI, Boy Scout Rule, SRP	01-
2	Naming & Comments — naming convention, function structure, magic numbers	02-
3	Refactoring & Estimation — refactoring patterns, PERT, planning poker, legacy code	03-
4	Code Review — checklist, constructive feedback, PR description, ESLint & Prettier	04-

Output Akhir Modul

Codebase Ter-refactor — ambil project kecil (todo app / calculator / utility CLI) lalu refactor total: bersihin naming, pisahin tanggung jawab, ilangin magic numbers, tambahin ESLint/Prettier, dan simulasi code review lewat PR.

AI Prompt Exercises

Sepanjang modul, latihan pake AI:

- "Review this code for DRY violations and suggest improvements"
- "Extract this function into smaller focused functions"
- "Rename these variables following clean code conventions"
- "Generate 5 test cases that would catch bugs in this refactored code"
- "Explain why this code violates the Single Responsibility Principle"

26.1 Clean Code Basics

Prinsip DRY (Don't Repeat Yourself)

Jangan tulis kode yang sama dua kali. Kalau ada pola berulang, bikin function/abstraksi.

```
// ❌ VIOLASI DRY – duplikasi logika validasi
function saveUser(name: string, email: string) {
  if (!name || name.length < 3) throw new Error("Nama tidak valid");
  if (!email.includes("@")) throw new Error("Email tidak valid");
  // save to DB...
}

function updateUser(name: string, email: string) {
  if (!name || name.length < 3) throw new Error("Nama tidak valid");
  if (!email.includes("@")) throw new Error("Email tidak valid");
  // update DB...
}

// ✅ DRY – extract validasi ke satu function
function validateName(name: string): void {
  if (!name || name.length < 3) throw new Error("Nama tidak valid");
}

function validateEmail(email: string): void {
  if (!email.includes("@")) throw new Error("Email tidak valid");
}
```

```

}

function saveUser(name: string, email: string) {
  validateName(name);
  validateEmail(email);
  // save to DB...
}

function updateUser(name: string, email: string) {
  validateName(name);
  validateEmail(email);
  // update DB...
}

```

Kapan Boleh Melanggar DRY?

- **Duplikasi yang tidak sengaja mirip** — dua function mirip secara kebetulan, beda domain. Abstraksi prematur bikin kode makin rumit.
- **Test code** — test boleh duplikasi setup demi readability. Gunakan factory function kalau setup terlalu panjang.
- **Config/boilerplate** — kadang duplikasi config lebih jelas daripada framework magic yang sulit di-debug.

Prinsip KISS (Keep It Simple, Stupid)

Kode paling sederhana adalah kode paling baik. Jangan over-engineer.

```

// □ OVER-ENGINEERED – class hierarchy tidak perlu
abstract class Greeter {
  abstract greet(): string;
}

class IndonesianGreeter extends Greeter {
  constructor(private name: string) { super(); }
  greet(): string {
    return `Halo, ${this.name}!`;
  }
}

class EnglishGreeter extends Greeter {
  constructor(private name: string) { super(); }
  greet(): string {

```

```

        return `Hello, ${this.name}!`;
    }
}

// ❏ KISS – function sederhana langsung
function greet(name: string, lang: "id" | "en"): string {
    const greetings = { id: "Halo", en: "Hello" };
    return `${greetings[lang]}, ${name}!`;
}

```

Ciri Kode KISS

- Bisa dibaca dalam sekali baca dari atas ke bawah
- Tidak ada class/abstraction yang tidak perlu
- Nama function jelas ngapain
- Satu function ngelakuin satu hal

Prinsip YAGNI (You Ain't Gonna Need It)

Jangan nambahin fitur/fungsi yang belum jelas bakal dipake. Tulis kode untuk kebutuhan *sekarang*, bukan kebutuhan imajiner.

```

// ❏ YAGNI VIOLATION – bikin generic repository padahal cuma pake User
interface Repository<T> {
    findAll(): Promise<T[]>;
    findById(id: string): Promise<T | null>;
    create(data: Partial<T>): Promise<T>;
    update(id: string, data: Partial<T>): Promise<T>;
    delete(id: string): Promise<void>;
    // ... 10 method lain yang belum kepake
}

class UserRepository implements Repository<User> {
    // implementasi 15 method...
    // tapi di app cuma kepake create() dan findAll()
}

// ❏ YAGNI – bikin spesifik dulu
interface UserRepository {
    findAll(): Promise<User[]>;
    create(data: CreateUserDto): Promise<User>;
}

class UserRepositoryImpl implements UserRepository {

```

```

async findAll(): Promise<User[]> {
    return db.query("SELECT * FROM users");
}
async create(data: CreateUserDto): Promise<User> {
    // ...
}
}

```

Tanda-tanda YAGNI:

- Interface dengan 10+ method cuma 2 kepake
 - Parameter opsional yang gak pernah dipake
 - Config flags untuk fitur yang belum ada
 - Abstraction "untuk jaga-jaga" tanpa use case konkret
-

Boy Scout Rule

"Selalu tinggalkan kode lebih bersih dari waktu kamu menemukannya."

Setiap kali menyentuh file, lakukan satu perbaikan kecil:

// SEBELUM – kode ditemukan dalam keadaan kotor

```

function calc(a, b, t) {
    let r = a * b;
    if (t == 1) return r + r * .1;
    if (t == 2) return r + r * .2;
    return r;
}

```

// SESUDAH – diperbaiki sedikit demi sedikit

```

type OrderType = "regular" | "express" | "priority";

```

```

const TAX_RATES: Record<OrderType, number> = {
    regular: 0.1,
    express: 0.2,
    priority: 0,
};

```

```

function calculateTotal(price: number, qty: number, type: OrderType): number {
    const subtotal = price * qty;
    const tax = subtotal * TAX_RATES[type];
    return subtotal + tax;
}

```

Perbaikan Kecil Tiap Sentuh:

- Rename variable a → price, t → type
 - Ganti magic number dengan constant
 - Tambah tipe
 - Hapus code yang di-comment
 - Fix indentasi
-

Single Responsibility Principle (SRP)

Satu function/class punya satu alasan untuk berubah.

// ❌ SRP VIOLATION – class ngurusin semuanya

```
class ReportService {
  generateReport(data: Sale[]) {
    // 1. hitung total
    const total = data.reduce((sum, item) => sum + item.amount, 0);
    // 2. format HTML
    const html = `<h1>Laporan</h1><p>Total: ${total}</p>`;
    // 3. simpan ke file
    fs.writeFileSync("report.html", html);
    // 4. kirim email
    email.send({ to: "admin@company.com", subject: "Laporan", html });
  }
}
```

// ✅ SRP – pisah tanggung jawab

```
class SalesCalculator {
  calculateTotal(data: Sale[]): number {
    return data.reduce((sum, item) => sum + item.amount, 0);
  }
}
```

```
class ReportRenderer {
  renderHtml(title: string, total: number): string {
    return `<h1>${title}</h1><p>Total: ${total}</p>`;
  }
}
```

```
class ReportRepository {
  save(filename: string, content: string): void {
    fs.writeFileSync(filename, content);
  }
}
```

```

class EmailService {
  sendReport(recipient: string, html: string): void {
    email.send({ to: recipient, subject: "Laporan", html });
  }
}

// Orchestrator
function generateAndSendReport(data: Sale[]) {
  const calculator = new SalesCalculator();
  const renderer = new ReportRenderer();
  const repo = new ReportRepository();
  const mailer = new EmailService();

  const total = calculator.calculateTotal(data);
  const html = renderer.renderHtml("Laporan Bulanan", total);
  repo.save("report.html", html);
  mailer.sendReport("admin@company.com", html);
}

```

Latihan

1. **DRY** — Kode berikut duplikasi logika diskon 3 kali di tempat berbeda. Refactor dengan extract function:

```

function checkoutItems(items: Item[]) {
  let total = items.reduce((sum, i) => sum + i.price, 0);
  if (total > 500000) total = total * 0.9;
  return total;
}

function applyPromo(items: Item[], code: string) {
  let total = items.reduce((sum, i) => sum + i.price, 0);
  if (total > 500000 && code === "PROMO10") total = total * 0.85;
  return total;
}

```

2. **KISS** — Sederhanakan kode ini tanpa mengubah behavior:

```

function getUserInfo(id: number): any {
  const d = fetch(`/api/users/${id}`);
  const j = d.json();
  const n = j.name;
  const e = j.email;
  const a = j.age;
  const r = { userName: n, userEmail: e, userAge: a, userType: j.type ?? "re" };
  return r;
}

```

```
}
```

3. **YAGNI** — Hapus parameter/code yang tidak diperlukan dari function ini (hanya 2 dari 5 parameter yang dipakai semua caller):

```
function renderPage(  
  title: string,  
  content: string,  
  footer?: boolean,  
  sidebar?: boolean,  
  theme?: "light" | "dark",  
  analytics?: boolean,  
  lang?: string  
): string { ... }
```

4. **SRP** — Class OrderProcessor handle: validasi, kalkulasi diskon, simpan ke DB, kirim email, log activity, generate invoice PDF. Pisah sesuai SRP.

26.2 Naming & Comments

Naming Convention

Variable — kata benda, jelas, hindari singkatan

```
// ❌ Buruk  
const d = new Date();  
const u = "Budi";  
const n = [1, 2, 3];  
const x = 0; // ini apaan?  
let flag = true; // flag apa?
```

```
// ✅ Baik  
const currentDate = new Date();  
const userName = "Budi";  
const scores = [1, 2, 3];  
const maxRetryAttempts = 3;  
let isUserLoggedIn = true;  
let isActive = true;
```

Function — kata kerja, deskriptif

```
// ❌ Buruk  
function data() { /* ... */ }  
function process() { /* ... */ }  
function handler() { /* ... */ }
```



```
function doStuff(a: number) { /* ... */ }

// ❌ Baik
function fetchUserId(id: number): Promise<User> { /* ... */ }
function calculateTotalPrice(items: Item[]): number { /* ... */ }
function validateEmail(email: string): boolean { /* ... */ }
function formatCurrency(amount: number): string { /* ... */ }
```

Boolean — beri predikat (is, has, can, should)

```
// ❌ Buruk
let visible = false; // visible apa?
let edit = true;     // edit mode? bisa diedit?
let dark = false;    // dark mode?

// ✅ Baik
let isVisible = false;
let isEditable = true;
let isDarkMode = false;
let hasPermission = true;
let canDelete = false;
let shouldShowWarning = true;
```

Class — kata benda, PascalCase

```
// ❌ Buruk
class user_service { /* ... */ }
class data { /* ... */ }
class helper { /* ... */ }

// ✅ Baik
class UserRepository { /* ... */ }
class InvoiceGenerator { /* ... */ }
class EmailSender { /* ... */ }
class DatabaseConnection { /* ... */ }
```

Constants — UPPER_SNAKE_CASE untuk nilai primitif yang benar-benar konstan

```
// ❌ Constant global
const MAX_FILE_SIZE = 10 * 1024 * 1024; // 10MB
const API_BASE_URL = "https://api.example.com/v2";
const TAX_RATE = 0.11;

// ❌ Bukan constant: dihitung runtime atau dari config
```

```
const config = loadConfig(); // const tapi bukan constant
const user = getUser();      // const tapi beda tiap request
```

Function Size & Structure

Aturan Praktis

- **Muat di layar** — ideal ≤ 20 baris, maksimal 40
- **Satu level abstraksi** — jangan campur high-level (business logic) dengan low-level (string manipulation)
- **Early return** — handle error case duluan, kurangi nesting

// ☐ Terlalu panjang, nested dalam, campur level abstraksi

```
function processOrder(order: Order): string {
  let result = "";
  if (order) {
    if (order.items && order.items.length > 0) {
      let total = 0;
      for (let i = 0; i < order.items.length; i++) {
        const item = order.items[i];
        const price = item.price * item.qty;
        total += price;
      }
      if (total > 0) {
        if (order.coupon) {
          total = applyCoupon(total, order.coupon);
        }
        result = `Total: ${formatRupiah(total)}`;
      } else {
        result = "Total tidak valid";
      }
    } else {
      result = "Order kosong";
    }
  } else {
    result = "Order tidak ditemukan";
  }
  return result;
}
```

// ☐ Refactor – early return, extract, satu level abstraksi

```
function processOrder(order: Order | null): string {
  if (!order) return "Order tidak ditemukan";
  if (!order.items?.length) return "Order kosong";
}
```

```

const total = calculateTotal(order.items);
if (total <= 0) return "Total tidak valid";

const finalTotal = order.coupon ? applyCoupon(total, order.coupon) : total;
return `Total: ${formatRupiah(finalTotal)}`;
}

function calculateTotal(items: Item[]): number {
  return items.reduce((sum, item) => sum + item.price * item.qty, 0);
}

```

Comments vs Code as Documentation

Komentar yang Baik

```

/**
 * Menghitung pajak PPN untuk transaksi e-commerce.
 * PPN 11% untuk transaksi di atas Rp100.000,
 * 0% untuk kebutuhan pokok (sesuai kategori).
 *
 * @param amount - nominal transaksi dalam Rupiah
 * @param category - kategori barang
 * @returns jumlah pajak yang harus dibayar
 */
function calculatePPN(amount: number, category: Category): number {
  if (amount <= 0) return 0;
  if (category.exemptFromTax) return 0;
  if (amount <= 100_000) return 0; // di bawah batas PPN
  return Math.round(amount * 0.11);
}

```

Komentar yang Seharusnya Kode

```

// □ Komentar menjelaskan "apa" – kode seharusnya sudah jelas
// Cek apakah user admin atau bukan
if (user.role === "admin" || user.role === "superadmin") { /* ... */ }

// □ Tulis ulang kode biar jelas tanpa komentar
const isAdmin = user.role === "admin" || user.role === "superadmin";
if (isAdmin) { /* ... */ }

// Atau pakai helper
function isAdminUser(user: User): boolean {

```

```

    return user.role === "admin" || user.role === "superadmin";
}

// ☐ Komentar karena function punya nama buruk
// apply diskon untuk member vip
function a(m: number, d: boolean): number {
    return d ? m * 0.8 : m;
}

// ☐ Perbaiki nama – komentar tidak perlu
function applyVipDiscount(price: number, isVipMember: boolean): number {
    return isVipMember ? price * 0.8 : price;
}

```

Kapan Komentar Diperlukan

- **Mengapa** — alasan keputusan teknis (bukan apa/cara)
- **API docs** — JSDoc untuk library/publik API
- **Gotcha / edge case** — kenapa handle case aneh ini
- **Referensi** — link ke ticket, spec, atau paper

```

// Kenapa pakai Math.round? Floating point di JS:
// 0.1 + 0.2 === 0.30000000000000004
// https://floating-point-gui.de/
const rounded = Math.round(amount * 100) / 100;

```

Magic Numbers Elimination

Magic number = angka mentah tanpa nama, bikin kode susah dipahami dan diubah.

```

// ☐ Magic numbers berserakan
function shippingCost(weight: number): number {
    if (weight <= 5) return 10000;
    if (weight <= 10) return 15000;
    if (weight <= 20) return 25000;
    return 50000;
}

function calculateLateFee(days: number): number {
    return days * 2000;
}

// ☐ Constant dengan nama jelas
const SHIPPING_TIER_1_MAX_KG = 5;

```

```

const SHIPPING_TIER_1_COST = 10_000;
const SHIPPING_TIER_2_MAX_KG = 10;
const SHIPPING_TIER_2_COST = 15_000;
const SHIPPING_TIER_3_MAX_KG = 20;
const SHIPPING_TIER_3_COST = 25_000;
const SHIPPING_TIER_4_COST = 50_000;

const DAILY_LATE_FEE = 2_000;

function shippingCost(weight: number): number {
  if (weight <= SHIPPING_TIER_1_MAX_KG) return SHIPPING_TIER_1_COST;
  if (weight <= SHIPPING_TIER_2_MAX_KG) return SHIPPING_TIER_2_COST;
  if (weight <= SHIPPING_TIER_3_MAX_KG) return SHIPPING_TIER_3_COST;
  return SHIPPING_TIER_4_COST;
}

function calculateLateFee(days: number): number {
  return days * DAILY_LATE_FEE;
}

```

Better: Struktur Data untuk Multi-Tier

```

// Saat ada banyak value terkait, pakai array of objects
const SHIPPING_TIERS = [
  { maxWeight: 5, cost: 10_000 },
  { maxWeight: 10, cost: 15_000 },
  { maxWeight: 20, cost: 25_000 },
  { maxWeight: Infinity, cost: 50_000 },
] as const;

function shippingCost(weight: number): number {
  const tier = SHIPPING_TIERS.find(t => weight <= t.maxWeight);
  return tier?.cost ?? 50_000;
}

```

Latihan

1. **Rename variable** — Perbaiki semua nama variable dan function dalam kode ini:

```

function p(a: number, b: number, c: string) {
  let d = a * b;
  if (c === "y") return d * 1.1;
  if (c === "m") return d * 1.2;
}

```

```
    return d;
}
```

2. **Magic numbers → constant** — Kode berikut punya 7 magic numbers (termasuk yang berulang). Eliminasi semua:

```
function calculatePension(age: number, yearsWorked: number, salary: number):
    if (age < 55) return 0;
    if (yearsWorked < 15) return salary * 0.5;
    if (yearsWorked < 25) return salary * 0.75;
    if (yearsWorked < 35) return salary * 0.9;
    return salary;
}
```

3. **Komentar → kode** — Refactor tanpa komentar, pake nama yang jelas:

```
// Check if user can access premium content
// by checking subscription status and age
function ck(u: any): boolean {
    let s = u.sub;
    let a = u.age;
    if (s === "premium" || s === "vip") {
        if (a >= 18) return true;
    }
    return false;
}
```

4. **Function size** — Pecah function 50+ baris ini jadi beberapa function kecil dengan single responsibility:

```
function handleUserRegistration(input: any) {
    // validasi input (15 baris)
    // cek duplicate email (8 baris)
    // hash password (5 baris)
    // save ke DB (10 baris)
    // generate token (5 baris)
    // kirim email verifikasi (10 baris)
    // return response (3 baris)
}
```

26.3 Refactoring & Estimation

Refactoring Patterns

Extract Function

Memecah blok kode besar menjadi function terpisah. Pattern paling dasar dan paling powerful.

```
// SEBELUM – satu function besar, terlalu banyak tanggung jawab
async function processInvoice(invoiceId: string): Promise<InvoiceResult> {
  const invoice = await db.invoices.findUnique({ where: { id: invoiceId } });
  if (!invoice) throw new Error("Invoice not found");

  if (invoice.status !== "pending") throw new Error("Invoice already processed")

  // Kalkulasi subtotal, diskon, pajak
  let subtotal = 0;
  for (const item of invoice.items) {
    subtotal += item.price * item.qty;
  }
  let discount = 0;
  if (subtotal > 1_000_000) discount = subtotal * 0.1;
  if (invoice.coupon === "SALE50") discount = Math.max(discount, subtotal * 0.5);
  const tax = (subtotal - discount) * 0.11;
  const total = subtotal - discount + tax;

  // Simpan ke DB
  await db.invoices.update({
    where: { id: invoiceId },
    data: { subtotal, discount, tax, total, status: "processed" },
  });

  // Kirim email
  await sendEmail({
    to: invoice.customerEmail,
    subject: `Invoice ${invoice.number} – Processed`,
    html: `

Total: Rp${total.toLocaleString()}</p>`,
  });

  return { invoiceId, subtotal, discount, tax, total, status: "processed" };
}

// SESUDAH – extract function
async function processInvoice(invoiceId: string): Promise<InvoiceResult> {
  const invoice = await findInvoiceOrThrow(invoiceId);


```

```

    assertInvoicePending(invoice);

    const { subtotal, discount, tax, total } = calculateInvoice(invoice);
    await updateInvoiceStatus(invoiceId, { subtotal, discount, tax, total });
    await notifyCustomer(invoice, total);

    return { invoiceId, subtotal, discount, tax, total, status: "processed" };
}

async function findInvoiceOrThrow(id: string): Promise<Invoice> {
    const invoice = await db.invoices.findUnique({ where: { id } });
    if (!invoice) throw new Error("Invoice not found");
    return invoice;
}

function assertInvoicePending(invoice: Invoice): void {
    if (invoice.status !== "pending") throw new Error("Invoice already processed");
}

function calculateInvoice(invoice: Invoice) {
    const subtotal = invoice.items.reduce((sum, item) => sum + item.price * item.qty, 0);
    const discount = calculateDiscount(subtotal, invoice.coupon);
    const tax = Math.round((subtotal - discount) * 0.11);
    const total = subtotal - discount + tax;
    return { subtotal, discount, tax, total };
}

function calculateDiscount(subtotal: number, coupon?: string): number {
    let discount = subtotal > 1_000_000 ? subtotal * 0.1 : 0;
    if (coupon === "SALE50") discount = Math.max(discount, subtotal * 0.5);
    return discount;
}

```

Rename — jangan takut rename

```

// SEBELUM — nama tidak jelas
function getData(id: string) {
    return db.query(`SELECT * FROM t WHERE id = $1`, [id]);
}

// SESUDAH — rename sesuai domain
function findUserById(id: string): Promise<User | null> {
    return db.query(`SELECT * FROM users WHERE id = $1`, [id]);
}

```



```
// Rename variable
const arr = [10, 20, 30]; // []
const temperatures = [10, 20, 30]; // []

const res = await fetch(url); // []
const response = await fetch(url); // []
```

Decompose Conditional

Ubah kondisi kompleks menjadi function bernama.

```
// SEBELUM – kondisi complex di dalam if
function calculateDiscount(price: number, user: User, order: Order): number {
  if (
    (user.role === "premium" || user.role === "vip") &&
    user.registrationDate < new Date("2024-01-01") &&
    order.items.length >= 3 &&
    order.totalAmount > 200000 &&
    !user.hasFlag("abuse")
  ) {
    return price * 0.2;
  }
  return 0;
}

// SESUDAH – setiap kondisi jadi function
function calculateDiscount(price: number, user: User, order: Order): number {
  if (isEligibleForPremiumDiscount(user, order)) {
    return price * 0.2;
  }
  return 0;
}

function isEligibleForPremiumDiscount(user: User, order: Order): boolean {
  return (
    isPremiumMember(user) &&
    isLoyalCustomer(user) &&
    meetsMinimumOrder(order) &&
    !hasAbuseFlag(user)
  );
}

function isPremiumMember(user: User): boolean {
  return user.role === "premium" || user.role === "vip";
}
```

```

function isLoyalCustomer(user: User): boolean {
    return user.registrationDate < new Date("2024-01-01");
}

function meetsMinimumOrder(order: Order): boolean {
    return order.items.length >= 3 && order.totalAmount > 200_000;
}

function hasAbuseFlag(user: User): boolean {
    return user.hasFlag("abuse");
}

```

Extract Class / Module

Ketika satu function/class terlalu banyak parameter atau terlalu besar.

```

// SEBELUM – satu file 400 baris
function sendNotification(user: User, type: string) {
    // logic email
    // logic SMS
    // logic push notification
    // logic in-app notification
    // formatting pesan
    // template rendering
}

// SESUDAH – modular
// notifications/email.service.ts
export class EmailNotificationService { /* ... */ }

// notifications/sms.service.ts
export class SMSNotificationService { /* ... */ }

// notifications/push.service.ts
export class PushNotificationService { /* ... */ }

// notifications/factory.ts
export function createNotificationService(type: NotificationType) { /* ... */ }

```

Estimation Techniques

PERT (Program Evaluation and Review Technique)

Rumus: **Estimate = (Optimistic + 4 × Most Likely + Pessimistic) / 6**

```

function pertEstimate(
  optimistic: number,
  mostLikely: number,
  pessimistic: number
): number {
  return (optimistic + 4 * mostLikely + pessimistic) / 6;
}

// Contoh: bikin fitur login
// Optimistic: 2 hari (semua lancar)
// Most likely: 4 hari
// Pessimistic: 10 hari (banyak bug & revisi)
console.log(pertEstimate(2, 4, 10)); // ~4.67 hari

// Standar deviasi
function pertStdDev(pessimistic: number, optimistic: number): number {
  return (pessimistic - optimistic) / 6;
}
// 90% yakin selesai dalam: estimate + (1.645 × stdDev)

```

Planning Poker

Teknik estimasi tim pakai kartu Fibonacci (1, 2, 3, 5, 8, 13, 21, 34, ?, ∞).

Langkah:

1. Product owner jelaskan user story
2. Setiap anggota tim pilih kartu secara rahasia
3. Semua buka kartu bersamaan
4. Kalau ada perbedaan besar (misal 2 vs 21), diskusi kenapa
5. Voting ulang sampai mendekati konsensus

Reference Task Method

Bandungkan task baru dengan task yang sudah pernah dikerjakan.

```

// Buat reference task database
const REFERENCE_TASKS = {
  "login-form": { hours: 4, complexity: "low" },
  "crud-users": { hours: 16, complexity: "medium" },
  "payment-integration": { hours: 40, complexity: "high" },
  "export-pdf": { hours: 12, complexity: "medium" },
} as const;

function estimateFromReference(
  taskDescription: string,

```

```

    similarTask: keyof typeof REFERENCE_TASKS,
    complexityDiff: number // multiplier: 1 = sama, 2 = dua kali lebih complex
  ): number {
    const reference = REFERENCE_TASKS[similarTask];
    return Math.round(reference.hours * complexityDiff);
  }

  // Task baru: "export Excel report" mirip "export-pdf", slightly lebih sederhana
  console.log(estimateFromReference("export-excel", "export-pdf", 0.8));
  // ~10 jam

```

Legacy Code Approach

Strategi kerja dengan kode legacy (tanpa test, tanpa dokumentasi):

1. Characterization Test

Tulis test yang menangkap behavior *sekarang* sebelum refactor.

```

describe("legacy calculateScore", () => {
  it("menghasilkan output yang sama setelah refactor", () => {
    // Arrange - bikin input sample
    const input = { a: 10, b: "x", c: true };

    // Act - jalanin legacy
    const legacyResult = calculateScoreLegacy(input.a, input.b, input.c);

    // Act - jalanin kode baru
    const newResult = calculateScore(input.a, input.b, input.c);

    // Assert - harus identik
    expect(newResult).toBe(legacyResult);
  });
});

```

2. Strangler Fig Pattern

Ganti komponen lama sedikit demi sedikit tanpa big bang rewrite.

```

// Step 1: Tambah adapter, dua-duanya jalan
class UserService {
  find(id: string): User {
    // Legacy SQL query
    return oldDb.query(`SELECT * FROM users WHERE id = '${id}'`);
  }
}

```

```

    }
}

class UserServiceNew {
    find(id: string): User {
        // New repository
        return userRepo.findById(id);
    }
}

// Step 2: Feature flag, sebagian traffic pake baru
class UserServiceRouter {
    constructor(
        private legacy: UserService,
        private modern: UserServiceNew
    ) {}

    find(id: string): User {
        // Slowly ramp up: 10% -> 50% -> 100%
        return shouldUseModernService(id)
            ? this.modern.find(id)
            : this.legacy.find(id);
    }
}

```

3. Refactor First, Add Feature Later

Jangan tambah fitur baru ke kode kotor — bersihin dulu, *lalu* tambah fitur.

Latihan

1. **Extract Function** — Pecah function ini jadi 4+ function kecil bernama jelas:

```

function processPayment(order: Order, payment: Payment): string {
    if (!order || !payment) return "Invalid data";
    if (order.status !== "confirmed") return "Order not confirmed";
    if (payment.amount < order.total) return "Insufficient payment";
    const fee = payment.method === "credit_card" ? order.total * 0.025 : payment.amount;
    const netAmount = payment.amount - fee;
    db.orders.update({ where: { id: order.id }, data: { status: "paid", paidAt: new Date() } });
    db.transactions.create({ data: { orderId: order.id, amount: netAmount, method: payment.method } });
    sendEmail({ to: order.customerEmail, subject: "Payment received", text: `Total payment received: ${order.total}` });
}

```

```
    return "Payment successful";
}
```

2. **Decompose Conditional** — Ubah kondisi kompleks jadi function-function bernama:

```
function canRedeemVoucher(user: User, voucher: Voucher, order: Order): boolean {
  if (
    voucher.expiresAt > new Date() &&
    !user.banned &&
    user.points >= voucher.minPoints &&
    order.total >= voucher.minPurchase &&
    (voucher.maxUsage === null || voucher.usedCount < voucher.maxUsage) &&
    (voucher.userLimit === null || !voucher.redeemedBy.includes(user.id))
  ) {
    return true;
  }
  return false;
}
```

3. **PERT Estimation** — Diberikan data: Optimistic=3 hari, Most Likely=7 hari, Pessimistic=15 hari. Hitung PERT estimate dan 90% confidence interval.
4. **Legacy Refactor** — Berikut function 80 baris tanpa type, tanpa test, parameter any. Tulis characterization test dulu, lalu refactor:

```
function save(d: any) {
  let x = d.n;
  let y = d.a;
  let z = d.e;
  // ... 70 baris messy logic
  return { id: d.id, name: x, age: y, email: z };
}
```

26.4 Code Review

Code Review Checklist

Functional Correctness

- ☐ Apakah kode menyelesaikan requirement yang diminta?
- ☐ Apakah edge cases di-handle (empty, null, invalid input)?
- ☐ Apakah error handling sudah sesuai (try/catch, proper error types)?
- ☐ Apakah perubahan tidak nge-break flow yang existing?

Code Quality

- ☐ Apakah nama variable/function/class jelas?
- ☐ Apakah tidak ada magic numbers / hardcoded values?
- ☐ Apakah function cukup kecil dan single responsibility?
- ☐ Apakah tidak ada duplikasi kode (DRY)?
- ☐ Apakah kode mengikuti style guide tim (ESLint/Prettier)?

Maintainability

- ☐ Apakah test menyertakan perubahan? (unit test, integration test)
- ☐ Apakah ada komentar yang menjelaskan "mengapa" bukan "apa"?
- ☐ Apakah perubahan mudah di-revert?
- ☐ Apakah dependensi baru justified? (tidak nambah library untuk hal sederhana)

Security

- ☐ Apakah input tervalidasi dan tersanitasi? (SQL injection, XSS)
- ☐ Apakah tidak ada credential/token hardcoded?
- ☐ Apakah authorization dicek untuk setiap endpoint?
- ☐ Apakah rate limiting perlu ditambah?

Performance

- ☐ Apakah tidak ada N+1 query?
 - ☐ Apakah loop efisien? (tidak nested loop $O(n^2)$ kalau bisa $O(n)$)
 - ☐ Apakah resource (file handle, DB connection) di-close dengan benar?
-

Constructive Feedback

Sandwich Method

- + Positif: "Bagus, validasi inputnya lengkap dan rapi."
- Perbaikan: "Function ini 60 baris, mungkin bisa di-extract jadi 3-4 function kecil."
- + Motivasi: "Overall approachnya solid, minor refactor aja."

Format Feedback Efektif

```
// ❌ Feedback buruk (menyerang orang, terlalu umum, tanpa solusi)
// "Ini kode jelek."
// "Logic-nya salah."
// "Kok gitu sih?"

// ✅ Feedback konstruktif (spesifik, deskriptif, kasih saran)
/*
 * Issue: Function `handlePayment` melakukan 4 hal sekaligus
 * (validasi, kalkulasi, simpan DB, kirim email).
 *
 * Saran: Extract jadi 4 function terpisah sesuai SRP.
 * - validatePayment(order, payment)
 * - calculateNetAmount(payment)
 * - saveTransaction(order, netAmount)
 * - notifyCustomer(order, netAmount)
 *
 * Benefit: lebih mudah di-test, debug, dan reuse.
 */
```

Contoh Code Review Komentar

```
// REVIEW COMMENT: Nama function kurang deskriptif
// handleData() → bikin bingung. Mungkin → processRefund() ?

// REVIEW COMMENT: Magic number
// if (age > 55) → constant RETIREMENT_AGE = 55

// REVIEW COMMENT: N+1 query
// Ini panggil db.items.findMany() dalam loop. Pake JOIN atau batch query.

// REVIEW QUESTION: Clarify intent
// Apakah `status === 3` itu cancelled? Lebih baik pake enum.
```

Security & Performance Review: Case Studies

Security: SQL Injection

```
// ❌ RAW STRING CONCATENATION – SQL injection risk
async function getUser(name: string) {
  return db.query(`SELECT * FROM users WHERE name = '${name}'`);
}
```



```
// □ PARAMETERIZED QUERY
async function getUser(name: string) {
  return db.query(`SELECT * FROM users WHERE name = $1`, [name]);
}
```

Security: Stored XSS

```
// □ Langsung render user input tanpa sanitasi
function renderComment(comment: string): string {
  return `<div>${comment}</div>`;
}
```

```
// □ Sanitasi / escape HTML
function sanitizeHtml(input: string): string {
  return input
    .replace(/&/g, "&amp;")
    .replace(/</g, "&lt;")
    .replace(/>/g, "&gt;")
    .replace(/"/g, "&quot;")
    .replace(/'/g, "&#x27;");
}
```

```
function renderComment(comment: string): string {
  return `<div>${sanitizeHtml(comment)}</div>`;
}
```

Performance: N+1 Query

```
// □ N+1 queries – 1 query for orders + N queries for order items
async function getOrdersWithItems(userId: string) {
  const orders = await db.orders.findMany({ where: { userId } });
  for (const order of orders) {
    order.items = await db.items.findMany({ where: { orderId: order.id } });
  }
  return orders;
}
```

```
// □ Single query with JOIN or batch
async function getOrdersWithItems(userId: string) {
  return db.orders.findMany({
    where: { userId },
    include: { items: true }, // JOIN via Prisma/ORM
  });
}
```

Performance: Unnecessary Computation

```
// □ Hitung ulang tiap render
function getTotalItems(cart: Cart): number {
  return cart.items.reduce((sum, item) => sum + item.qty, 0);
}

// □ Cache / memoize kalau data tidak berubah
let cachedTotal: number | null = null;
let lastCartVersion = 0;

function getTotalItems(cart: Cart): number {
  if (cart.version !== lastCartVersion) {
    cachedTotal = cart.items.reduce((sum, item) => sum + item.qty, 0);
    lastCartVersion = cart.version;
  }
  return cachedTotal!;
}
```

PR Description Template

Description

[Explain what this PR does and why]

Related Ticket

Closes #[ticket-number]

Type of Change

- [] Bug fix
- [] New feature
- [] Refactor
- [] Documentation
- [] Performance

How Has This Been Tested?

- [] Unit tests
- [] Integration tests
- [] Manual testing

Screenshots (if applicable)

Checklist

- [] My code follows the project's style guidelines
- [] I have self-reviewed my own code

- [] I have added tests that prove my fix/feature works
- [] No new warnings
- [] Breaking changes documented

Additional Notes

[Anything reviewers should know]

Contoh PR Deskripsi Nyata

Description

Refactor `processInvoice()` – extract inline logic into focused functions per SRP. No behavior change; all existing tests pass.

Related Ticket

Closes #142 – Payment flow refactoring

Type of Change

- [x] Refactor

Checklist

- [x] Code follows style guide
 - [x] Self-reviewed
 - [x] All tests pass (see CI)
-

Tools: ESLint & Prettier

Setup ESLint + Prettier untuk TypeScript

```
npm init -y
npm install -D typescript @typescript-eslint/parser @typescript-eslint/eslint-plugin
npm install -D eslint prettier eslint-config-prettier eslint-plugin-prettier
```

```
// .eslintrc.json
```

```
{
  "parser": "@typescript-eslint/parser",
  "plugins": ["@typescript-eslint", "prettier"],
  "extends": [
    "eslint:recommended",
    "plugin:@typescript-eslint/recommended",
    "plugin:prettier/recommended"
  ],
  "rules": {
    "@typescript-eslint/no-explicit-any": "warn",
    "@typescript-eslint/no-unused-vars": "error",
```

```

    "no-console": "warn",
    "prettier/prettier": [
      "error",
      { "semi": true, "singleQuote": false, "tabWidth": 2, "trailingComma": "all" }
    ]
  }
}
// .prettierrc
{
  "semi": true,
  "singleQuote": false,
  "tabWidth": 2,
  "trailingComma": "all",
  "printWidth": 100
}

```

Rules yang Paling Berguna

Rule	Efek
no-explicit-any	Larang any, paksa pake tipe yang proper
no-unused-vars	Tangkap variable nganggur
no-console	Cegah console.log masuk production
no-magic-numbers	Tangkap magic numbers
max-lines-per-function	Batasi panjang function
prefer-const	Paksa pake const kalau tidak di-reassign

Integrasi Git Hook (Husky)

npm install -D husky lint-staged

```

// package.json tambahkan:
"lint-staged": {
  "/*.ts": ["eslint --fix", "prettier --write"]
}

```

Latihan

1. **Code Review Simulation** — Review PR berikut. Kasih 3 feedback konstruktif (1 positif, 2 improvement):

```

// src/services/user.service.ts
// PR: Add user registration endpoint

```

```

async function reg(a: any, b: any) {
  if (!a || !b) {
    return 0;
  }
  const x = await db.query(`SELECT * FROM users WHERE email = '${b}'`);
  if (x.length > 0) {
    return "exists";
  }
  const y = await db.query(`INSERT INTO users(name,email) VALUES('${a}','${b}')`);
  sendEmail(b, "Welcome");
  return y;
}

```

2. **Security Review** — Temukan 3 security issues dalam kode berikut dan perbaiki:

```

async function login(req: any, res: any) {
  const { username, password } = req.body;
  const user = await db.query(`SELECT * FROM users WHERE username = '${username}'`);
  if (user[0].password === password) {
    res.send({ token: "secret-token-" + user[0].id });
  }
  res.send({ error: "Login failed" });
}

```

3. **Performance Review** — Temukan performance issue dan refactor:

```

async function getUserDashboard(userId: string) {
  const user = await db.users.findOne({ id: userId });
  const posts = await db.posts.find({ authorId: userId });
  for (const post of posts) {
    post.comments = await db.comments.find({ postId: post.id });
    for (const comment of post.comments) {
      comment.likes = await db.likes.count({ commentId: comment.id });
    }
  }
  return { user, posts };
}

```

4. **PR Description** — Tulis PR description lengkap untuk refactoring function berikut (gunakan template PR di atas):

```

// Before – monolithic
function handleOrder(o: any) { /* 150 baris: validasi, payment, stock, email */ }

// After – refactored into 5 focused functions

```